Clase 16

MUTACIÓN

Mutación

- Hasta ahora, todas nuestras funciones reciben parámetros y retornan un valor de acuerdo a éstos
  - Para los mismos parámetros se devuelve el mismo valor
  - Esto facilita muchísimo hacer testing

Mutación

- Sin embargo, hay programas que requieren que una función pueda tener una cierta *memoria* sobre las acciones sobre los datos
  - Agenda telefónica: suponga que Ana tiene el numero 5551111

```
telefonoAna = consultarTelefono('Ana') # devuelve 5551111
```

  - Ana cambia su número a 5551112

```
telefonoAna = consultarTelefono('Ana') # devuelve 5551112
```


Mutación

- Disponer de *memoria* puede ser muy útil
  - Servicios al usuario para administrar información
- Pero, la programación es más compleja y requiere ser cuidadoso
  - Ya no es fácil hacer testing (no conocemos a priori la respuesta correcta)
  - Un cambio realizado a la *memoria* puede tener impacto en distintas partes del programa

Mutación

- Variables de estado: variable que implementa *memoria*
- En Python, una variable de estado se implementa como una variable global
  - Se puede acceder desde cualquier función y desde el programa principal
  - Su valor puede **mutar** (actualizarse)

Mutación

- Ambiente de una variable (*scope*): es la parte del código en donde una variable puede ser accesada
  - Variables locales:
    - Se definen dentro de una función
    - Su *scope* es la función: una vez que termina la función, desaparece la variable
  - Variables de estado (o globales):
    - Se definen a nivel de la línea de comando
    - Su *scope* es el programa completo

Mutación

- Definición y uso de variable de estado:

```
# Variable de estado
contador = 0

# procedimiento: None -> None
# efecto: modifica variable contador incrementando su valor en 1
def procedimiento():
    global contador
    contador = contador + 1

# Programa principal
print(str(contador))
procedimiento()
print(str(contador))
procedimiento()
procedimiento()
procedimiento()
print(str(contador))
```


Mutación

- Ejemplo: Implementar una agenda telefónica con las siguientes operaciones

```
import estructura

# registro: nombre(str) numero(int)
estructura.crear('registro', 'nombre numero')

# buscar: str -> int
# devuelve el numero de telefono asociado a nombre o -1 si nombre no esta

# agregar: registro -> None
# efecto: agrega registro a la agenda

# borrar: registro -> None
# efecto: borra de la agenda el registro
```

Mutación

- Hay que decidir qué utilizar para implementar la memoria
  - Usaremos una lista de registros

```
from lista import *

# registro: nombre(str) numero(int)
estructura.crear('registro', 'nombre numero')

# Variable de estado
contactos = listaVacia
```


Mutación

- Función buscar:

```
# buscar: str -> int
# devuelve el numero de telefono asociado a nombre o -1 si nombre no esta
def buscar(nombre):
    global contactos
    return buscarRec(nombre, contactos)

# buscarRec: str lista(registro) -> int
# devuelve el numero de telefono de nombre o -1 si no esta en unaLista
def buscarRec(nombre, unaLista):
    if vacia(unaLista):
        return -1
    elif cabeza(unaLista).nombre == nombre:
        return cabeza(unaLista).numero
    else:
        return buscarRec(nombre, cola(unaLista))
```

Mutación

- Función agregar:

```
# agregar: registro -> None
# efecto: agrega registro persona al principio de variable contactos
def agregar(persona):
    global contactos
    contactos = lista(persona, contactos)
```

- Función borrar:

```
from funcionesAbstractasListas import filtro

# borrar: registro -> None
# efecto: borrar registros en variable contactos con mismos atributos que
# persona
def borrar(persona):
    global contactos
    contactos = filtro(lambda x: x != persona, contactos)
```


Mutación

- ¿Cómo hacer testing?
  - Definir un estado conocido a la(s) variable(s) de estado
    - Sugerencia: utilizar un procedimiento que inicialize variables de estado
  - Invocar las funciones que se están probando y comprobar que, para el estado definido, devuelven la respuesta correcta y/o modifican la variable de estado en la forma esperada

Mutación

- Testing para funciones buscar y agregar:

```
# inicializador: None -> None
# efecto: inicializa la variable de estado contactos, dejandola vacia
def inicializador():
    global contactos
    contactos = listaVacia

# Test de todas las funciones,
inicializador() # con esto, el estado inicial de contactos es conocido
pedro = registro('Pedro', 5553856)
juan = registro('Juan', 5552946)
diego = registro('Diego', 5553386)
isabel = registro('Isabel', 5550743)
agregar(pedro)
agregar(isabel)
assert buscar(pedro.nombre) == pedro.numero
assert buscar(isabel.nombre) == isabel.numero
assert buscar(diego.nombre) == -1
agregar(diego)
agregar(juan)
assert buscar(diego.nombre) == diego.numero
```


Para la próxima clase

- Leer los Capítulos 11.3 y 11.4 del apunte